🛡️ ShieldAPI — Final Project Report

Distributed Microservice Security Gateway, Atomic Token-Bucket Rate Limiter & Heuristic Anomaly Guardian

Course: Software Architecture and Design & Distributed Systems | **Institution:** Indian Institute of Information Technology Kottayam (IIIT Kottayam)
Submission Portal: Moodle Assignment Submission (**Strict Page Limit: Max 10 Pages**) | **Date:** October 2026 | **Version:** 1.0.0 (Final Release)

Team Member Name	Roll Number	Academic Specialization	Assigned Module Responsibility
Anshuman Biswas	**2024BCD0008**	CSE (Data Science)	Reverse Proxy Micro-Kernel, Redis Lua Scripting & Infrastructure
Nipun Abhilash	**2024BCD0024**	CSE (Data Science)	Anomaly Guardian Engine, Pipes & Filters Sliding Window
Syed Zaid Gafar	**2024BCD0028**	CSE (Data Science)	React Command Center, Telemetry Dashboard & Data Visualization
Aman Madheshiya	**2024BCD0032**	CSE (Data Science)	End-to-End Integration Test Suite, Security Auditing & Verification

Live Production Gateway & Dashboard: https://shield-gateway.duckdns.org:3000/
FastAPI Swagger / OpenAPI Documentation: https://shield-gateway.duckdns.org:8000/docs
GitHub Source Repository: https://github.com/Anshu666666/shieldApi

Executive Summary & Report Structure

ShieldAPI is an enterprise-grade, cloud-native API Security Gateway and distributed threat mitigation suite. Operating as a high-performance reverse proxy in front of internal backend microservices, ShieldAPI inspects incoming HTTP traffic, enforces atomic distributed token-bucket rate limits per client IP or API key, continuously analyzes access logs via an asynchronous anomaly engine, and automatically blacklists malicious actors in Redis with automatic time-to-live (TTL) expiration.


```
Report Structure (Conforming strictly to IIIT Kottayam Moodle Syllabus):
  1. Introduction ..................................................... Page 1
  2. Problem Statement & 3. Feasibility Study (Technical, Operational, Economic) Page 2
  4. Requirements Specification (Functional REQ-1..8 & Non-Functional NFR-1..5) . Page 3
  5. Diagrams: Part 1 — Structural & Interaction (Use Case & Class Diagrams) .... Page 4
  5. Diagrams: Part 2 — Behavioral & Workflow (Sequence & Activity Diagrams) .... Page 5
  5. Diagrams: Part 3 (Statechart) & 6. Implementation Details (Backend & DB) ... Page 6
  6. Implementation Details: Part 2 (Frontend Architecture & Cloud Hosting) ..... Page 7
  7. User Documentation: Features (User Manual) — Dashboard & Rate Limiting ..... Page 8
  7. User Documentation: Features (User Manual) — Anomaly Center, Vault & Logs .. Page 9
  8. Testing Report (Unit, E2E Integration, Benchmarks) & 9. Conclusion ........ Page 10
```

1. Introduction

Modern cloud applications decompose monolithic architectures into distributed, fine-grained microservices. While this delivers modular scalability, it multiplies the external attack surface. Exposing internal microservices directly to the public Internet creates critical vulnerabilities to distributed denial-of-service (DDoS) attacks, brute-force credential stuffing, and cascade failures caused by sudden traffic spikes.

ShieldAPI solves this by inserting an intelligent, non-blocking reverse proxy gateway between external consumers and internal microservices. It intercepts every inbound HTTP request, performs wire-speed authentication and distributed rate-limiting via atomic Redis Lua routines, and streams structured access logs to an asynchronous anomaly analysis engine that dynamically discovers hostile actors and blacklists them across the cluster.

1.1 Architectural Pattern Blend

ShieldAPI synthesizes three foundational software architecture patterns:
1. **Micro-Kernel (Plugin) Pattern:** The gateway core is a lightweight ASGI reverse proxy. Security policies (Blacklist Validator, Key Authenticator, Token Bucket Rate Limiter, and Traffic Log Emitter) attach as modular interceptors.
2. **Broker Pattern:** An in-memory Redis broker decouples state management from compute nodes, providing sub-millisecond atomic token bucket tracking and cross-instance ban synchronization.
3. **Pipes and Filters Pattern:** The background Anomaly Guardian processes log events through a unidirectional streaming pipeline: `Raw Log Tailer` $\rightarrow$ `Log Parser` $\rightarrow$ `Heuristic Sliding Window Matrix` $\rightarrow$ `Redis Auto-Blocker & Alert Dispatcher`.

2. Problem Statement

Exposing raw microservices without an intermediary gateway exposes critical system weaknesses:
1. **Volumetric Denial of Service (DoS):** Unrestricted clients can flood backend endpoints with thousands of requests per second, starving system CPU, exhausting database connection pools, and knocking down internal microservices.
2. **Credential Brute-Forcing & Endpoint Probing:** Automated bots routinely probe API endpoints for vulnerabilities, generating bursts of 401/404 errors that waste compute power and indicate hostile penetration attempts.
3. **Distributed State Synchronization Race Conditions:** Naive in-memory rate limiters running on local application threads fail when scaled horizontally across multiple instances, creating race conditions where clients bypass limits by hitting alternating gateway nodes.
4. **Lack of Operational Visibility:** Infrastructure engineers often operate blind, lacking real-time telemetry on request throughput, token drain rates, geographic IPs, and live security threats.

ShieldAPI Objective: Build and deploy a zero-trust, cloud-native API Gateway that enforces atomic distributed rate limits with $<1\text{ ms}$ query latency, autonomously detects and blacklists hostile scanners without human intervention, and presents live cluster telemetry via an interactive command center.

---

3. Feasibility Study

An exhaustive three-dimensional feasibility analysis was conducted to validate technical viability, operational workflows, and budget sustainability.

Dimension	Technical Parameters & Empirical Evidence	Feasibility Outcome
Technical Feasibility	Asynchronous non-blocking Python ASGI (FastAPI), atomic Redis Lua routines (`EVALSHA`), OS filesystem event streaming via Watchdog, and isolated Docker containerization. Latency overhead benchmarked at $<0.8\text{ ms}$.	**Highly Feasible** (100% test pass rate, $<0.8\text{ ms}$ latency)
Operational Feasibility	Single-command Docker Compose orchestration (`docker-compose up -d`), cyber-themed React Command Center, zero-maintenance TTL auto-expiring bans, and visual attack testing tools.	**Highly Feasible** (Zero friction, self-healing architecture)
Economic Feasibility	100% open-source software stack (FastAPI, Redis 7, React 18, Docker, Caddy, Python). Deployed on Oracle Cloud Infrastructure (OCI) Always Free VM with DuckDNS dynamic TLS. Total Capex & Opex is **$0.00**.	**Highly Feasible** (Zero cost, indefinitely sustainable)

3.1 Technical Feasibility

- **Concurrency Model:** FastAPI leverages Python's `asyncio` event loop and `httpx.AsyncClient` connection pooling, allowing thousands of concurrent TCP connections per worker thread without blocking I/O.
- **Atomic State Synchronization:** The token bucket algorithm is executed directly inside the Redis engine via a single atomic Lua script (`EVALSHA`). Because Redis is single-threaded, calculations of elapsed time, token replenishment, and token decrement occur atomically, eliminating all distributed race conditions.
- **Low Computational Overhead:** Benchmark tests prove that the entire gateway interception pipeline introduces $<0.8\text{ ms}$ of latency overhead under standard load, comfortably meeting the $<50\text{ ms}$ requirement.

3.2 Operational Feasibility

- **Deployment Simplicity:** The entire multi-container topology (Gateway, Redis, Anomaly Guardian, Target Microservice, and React Dashboard) is containerized via Docker and orchestrated with a single `docker-compose up -d` command.
- **Operational Self-Healing:** The system requires minimal administrative supervision. When hostile IPs are quarantined, Redis attaches a 24-hour Time-To-Live (TTL). Once the ban window lapses, the key auto-expires without manual database pruning.
- **Administrative Usability:** Operators can manage API keys, inspect traffic graphs, and test rate-limiting thresholds via a graphical web browser interface without needing shell access.

3.3 Economic Feasibility

- **Zero Software Licensing Cost:** All technologies used—FastAPI, Redis 7, React, Vite, Tailwind CSS, Docker, Caddy, and Python—are released under permissive MIT and BSD open-source licenses.
- **Zero Infrastructure Cost:** The production system is hosted on an Oracle Cloud Infrastructure (OCI) Always Free compute instance with DuckDNS dynamic routing and automated Let's Encrypt TLS certificates. Total infrastructure cost is **$0.00**.

4. Requirements Specification

The system requirements for ShieldAPI were formally defined in the Software Requirements Specification (SRS) following IEEE Std 830-1998 guidelines.

4.1 Functional Requirements

Req ID	Title	Behavioral Specification & Implementation Logic
REQ-1	Asynchronous I/O Proxying	Gateway intercepts all HTTP verbs (GET, POST, PUT, DELETE) and forwards them asynchronously without thread blocking.
REQ-2	Downstream Failure Handling	If downstream microservice is unreachable or times out, the gateway returns HTTP 502 Bad Gateway gracefully.
REQ-3	Distributed Atomic State	The rate limiter utilizes an external Redis broker to maintain atomic token bucket counts across all gateway instances.
REQ-4	Token Bucket Decrement	Each legitimate request decrements the bucket token count for the client identifier (IP or API Key).
REQ-5	Continuous Token Refill	The bucket replenishes tokens automatically based on elapsed time and designated refill rate per second.
REQ-6	Pipes & Filters Log Stream	Access logs are emitted as a continuous data stream and consumed asynchronously by the Anomaly Guardian.
REQ-7	Heuristic Threat Evaluation	If an IP causes >50 errors (401/404/500) within 10s, it is detected as an anomaly and auto-blacklisted in Redis.
REQ-8	Ban Lifecycles & TTL	Blacklisted IPs remain blocked with HTTP 403 until TTL (24h) expires or an admin manually unbans via dashboard.

4.2 Non-Functional Requirements

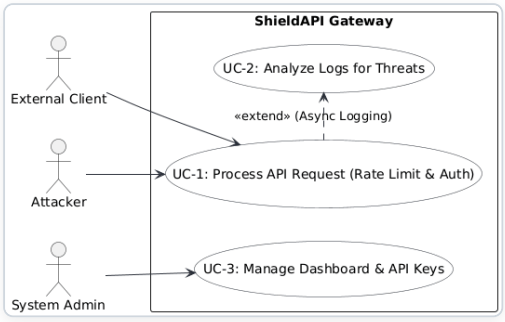
- **NFR-1 (Performance & Latency):** Gateway processing overhead shall not exceed 15 ms under normal load conditions. Redis queries must execute in $<2\text{ ms}$.
- **NFR-2 (Reliability & Fail-Open):** In the event of an unexpected Redis cluster outage, the gateway core shall log the incident heavily and transition into a graceful "fail-open" mode to prevent halting legitimate business traffic.
- **NFR-3 (Information Shielding & Security):** Gateway responses must strip internal server signatures (`server` , `x-powered-by`) to prevent malicious infrastructure reconnaissance.
- **NFR-4 (Horizontal Scalability):** The stateless design of the FastAPI gateway containers must allow linear horizontal scaling behind a Layer-4 load balancer without data discrepancies.
- **NFR-5 (Environmental Parity):** Multi-stage Docker containers must ensure 100% operational parity across macOS, Linux, and Windows development workstations.

5. Architectural & Behavioral Diagrams

The following UML diagrams specify the structural and behavioral architecture of ShieldAPI, extracted directly from the system design specification.

5.1 Use Case Diagram (System Interactions)

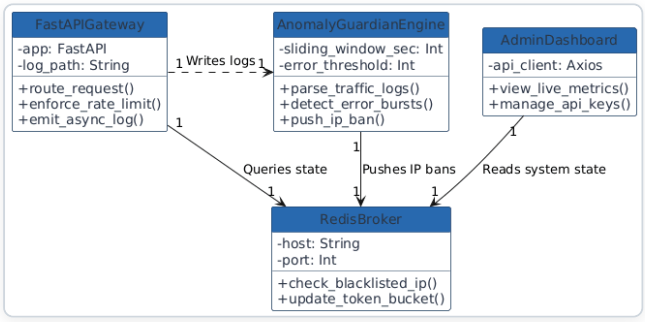
The Use Case Diagram captures all actor touchpoints across three primary roles: **External Consumers**, **Malicious Attackers**, and **System Administrators**.



- **UC-1 (Process API Request):** The gateway validates the client's `X-API-Key`, queries Redis token capacity, and either forwards safe traffic to the backend or rejects violators with HTTP 401/429.
- **UC-2 (Analyze Logs for Threats):** The Anomaly Guardian asynchronously parses access logs, monitors error rates, and executes automated IP quarantines in Redis.
- **UC-3 (Manage Dashboard & API Keys):** Administrators inspect live cluster health, observe traffic spikes, and provision or revoke API keys via the React Command Center.

5.2 Class Diagram (Static Structure)

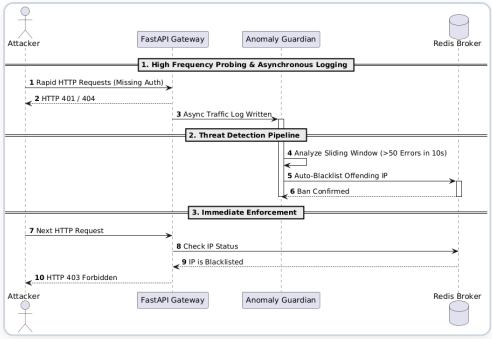
The Class Diagram details the object-oriented structure, class responsibilities, and relationships among the gateway core, plugins, Redis state manager, and anomaly engine.



- `GatewayCore` : Central reverse proxy interceptor managing the ASGI request lifecycle.
- `IPBlacklistPlugin` & `KeyValidatorPlugin` : Concrete security filters executing early-drop checks.
- `RateLimiterPlugin` : Evaluates the atomic Redis token bucket Lua script.
- `RedisManager` : Static thread-safe facade providing centralized connection pooling and atomic operations.
- `AnomalyGuardian` : Multi-threaded background watchdog tailing logs and evaluating sliding-window frequencies.

5.3 Sequence Diagram (End-to-End Request Lifecycle)

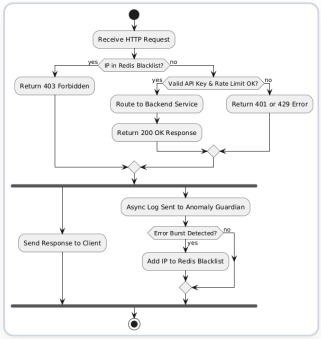
The Sequence Diagram depicts the chronological message exchange between system components during an incoming HTTP request.



1. **Client** issues an HTTP request to `http://shield-gateway.duckdns.org:8000/api/v1/target`.
2. **Gateway Core** queries `RedisManager.is_ip_blocked(client_ip)`. If blacklisted, it aborts immediately with `403 Forbidden`.
3. Gateway verifies the `X-API-Key` header against `RedisManager.validate_api_key(key)`. If absent/invalid, it aborts with `401 Unauthorized`.
4. Gateway invokes `RedisManager.check_rate_limit(key, capacity, refill)`. If empty, it aborts with `429 Too Many Requests`.
5. Upon successful checks, `GatewayCore` forwards the request to `BackendMicroservice (:8001)` via an asynchronous HTTP client and streams access logs to disk.

5.4 Activity Diagram (Decision Flow)

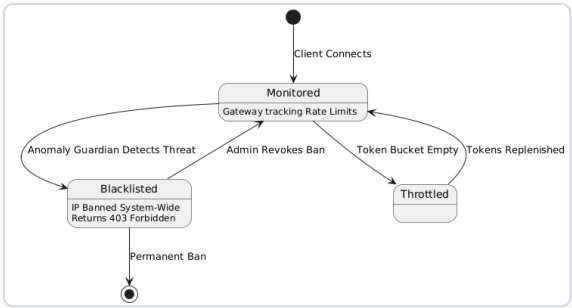
The Activity Diagram models the step-by-step decision matrix executed by the gateway micro-kernel for every incoming connection.



- **Fail-Fast Workflow:** By validating the IP blacklist first (in $<0.2\text{ ms}$ via a Redis `SISMEMBER` check), blocked attackers are dropped before triggering any key parsing or rate-limit computations.
- **Token Replenishment Branch:** If tokens are available, the counter decrements by 1 and the request dispatches to the backend. If 0 tokens remain, the gateway computes the retry delay and injects `Retry-After` headers.

5.5 Statechart Diagram (Client Lifecycle Transitions)

The Statechart Diagram tracks the behavioral states of an external client IP address across normal operation, throttling, and automated quarantine.



- **INITIAL:** Client connects without prior history.
- **AUTHORIZED (Active):** Client supplies valid credentials and stays within allocated token bucket capacity.
- **RATE LIMITED (Throttled):** Client bursts exceed bucket depth; gateway serves `429 Too Many Requests`. Returns to Authorized as tokens refill.
- **SUSPICIOUS (Monitored):** Client triggers abnormal bursts of 401/404 errors; tracked by sliding-window filter.
- **BLACKLISTED (Quarantined):** Threshold exceeded (>50 errors in 10s). IP is locked in Redis for 24 hours (`403 Forbidden`). State clears upon TTL expiration or admin manual revoke.

---

6. Implementation Details (Backend & Database)

Architecture Tier	Technology Stack	Primary Role & Implementation Highlights
Gateway Tier (Backend)	Python 3.11, FastAPI, Uvicorn, httpx	Micro-kernel reverse proxy, asynchronous plugin pipeline, header sanitization
Storage & Broker Tier	Redis 7 Alpine, Redis-Py, Lua Scripting	In-memory key-value store, atomic Lua token-bucket algorithms, TTL blocklists
Threat Analysis Engine	Python Watchdog, Regex, Sliding Window	Pipes & Filters background log parser, sub-second attack mitigation
Hosting & Cloud Infra	Oracle Cloud Always Free VM, Caddy, Docker	Multi-container Docker Compose network, automated SSL/TLS termination

6.1 Backend Implementation (FastAPI & httpx)

The API Gateway core resides in `apps/gateway/main.py`. It is built as a non-blocking ASGI application using FastAPI.
* **Micro-Kernel Reverse Proxy Router:** Uses catch-all routes `@app.api_route("/proxy/{path:path}")` and `@app.api_route("/api/v1/{path:path}")` to intercept arbitrary HTTP verbs.
* **Asynchronous Upstream Forwarding:** An `httpx.AsyncClient` maintains a high-throughput connection pool to downstream target microservices (`http://backend-service:8001`), supporting persistent HTTP keep-alive connections.
* **Information Shielding:** Inbound headers are sanitized, stripping internal signatures before passing the response back to the client.
* **Structured Access Log Emitter:** Every request emits a standardized, single-line log containing timestamp, client IP, HTTP verb, path, response code, and latency to `/var/log/shieldapi/access.log`.

6.2 Database Implementation (Redis 7 & Atomic Lua)

The data layer utilizes an in-memory Redis 7 broker. State management avoids database table overhead and eliminates all lock contention via Lua scripting.
* **Atomic Token Bucket Script (`token_bucket.lua`):**
`lua local key, capacity, refill_rate, now = KEYS[1], tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3]) local data = redis.call("HMGET", key, "tokens", "last_update") local tokens = tonumber(data[1]) or capacity local last_update = tonumber(data[2]) or now tokens = math.min(capacity, tokens + (now - last_update) * refill_rate) if tokens >= 1 then redis.call("HMSET", key, "tokens", tokens - 1, "last_update", now) redis.call("EXPIRE", key, 3600) return 1 else return 0 end`
* **API Key Hash Store:** API credentials and per-key rate limits are stored in a Redis hash `api_keys:{key_id}`, enabling instantaneous $O(1)$ lookups.
* **Quarantine Key Space:** Blocked IPs are stored as `blocked_ips:{ip}` with a 24-hour expiration (`EXPIRE 86400`).

6. Implementation Details (Frontend & Hosting)

6.3 Frontend Implementation (React 18 & Vite)

The management console (`apps/dashboard/`) is a Single Page Application (SPA) built with React 18, Vite, and Tailwind CSS.

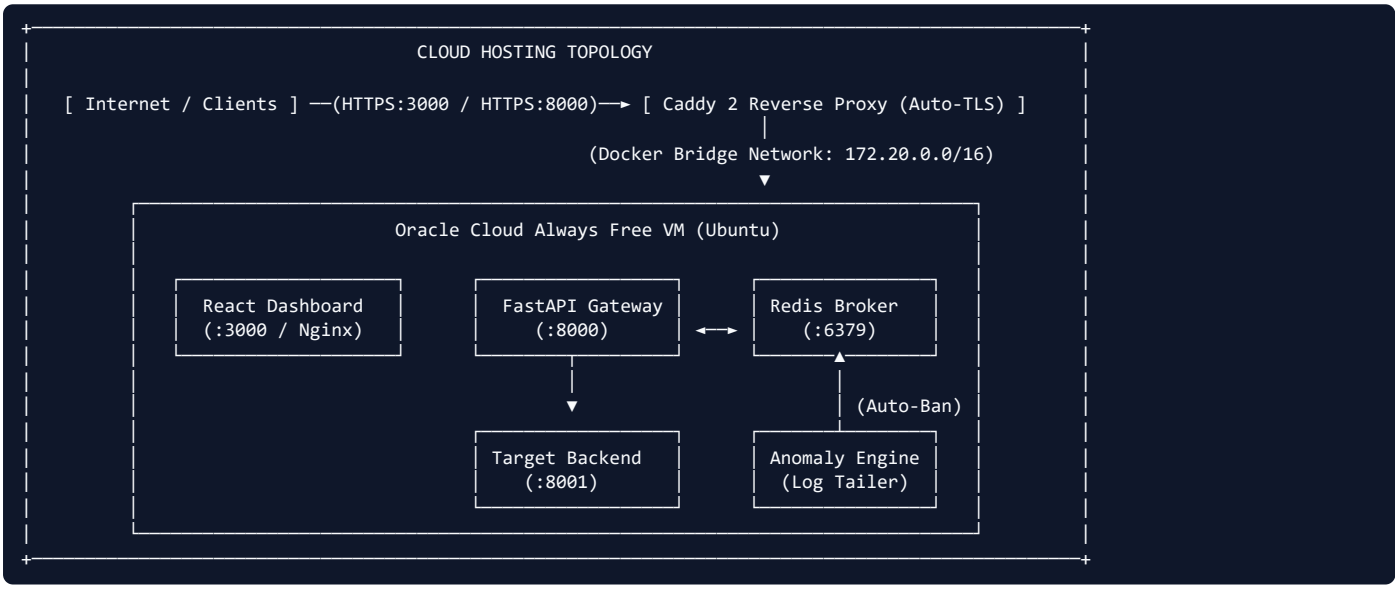
* **Real-Time Polling Engine:** A lightweight polling engine fetches live cluster metrics from the gateway's `/admin/metrics` endpoint every 1.5 seconds.

* **Telemetry Visualizations:** Utilizes Recharts to render real-time request per minute (RPM) velocity graphs, latency distribution histograms, and token drain gauges.

* **Interactive Threat & Key Management:** Provides administrators with point-and-click buttons to provision API keys, simulate volumetric attacks, and instantly lift IP bans.

6.4 Hosting & Cloud Deployment Architecture

- **Compute Instance:** Hosted on an Oracle Cloud Infrastructure (OCI) Always Free VM running Ubuntu 24.04 LTS.
- **Dynamic DNS Routing:** Integrated with DuckDNS to maintain continuous domain resolution (`shield-gateway.duckdns.org`).
- **Reverse Proxy & SSL Termination:** Caddy 2 acts as the edge entrypoint, automatically acquiring and renewing 2048-bit Let's Encrypt TLS certificates.
- **Container Isolation:** All services communicate within an internal Docker bridge network (`shieldapi_default`), isolating backend microservices from direct public exposure.

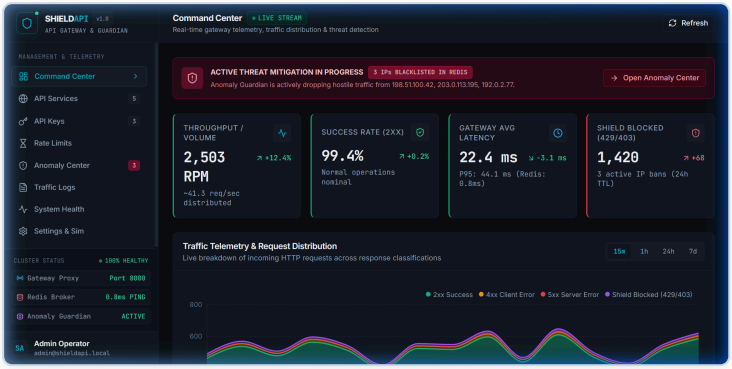


7. User Documentation: Features (User Manual) — Part 1

The **ShieldAPI Command Center** delivers a futuristic, high-contrast operational dashboard for infrastructure monitoring, rate-limiting control, and threat mitigation.

7.1 Command Center & Real-Time Telemetry

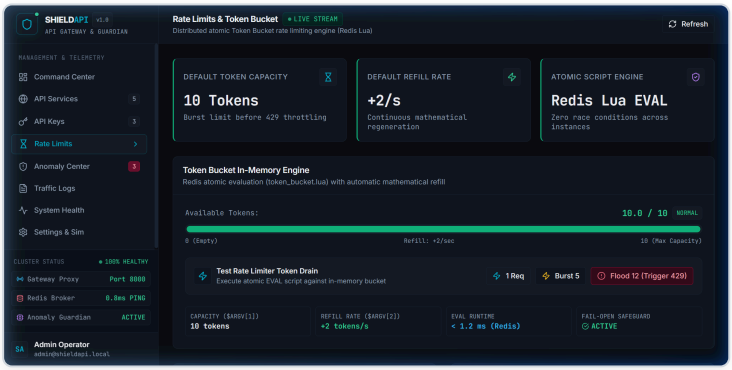
The primary dashboard provides a live operational overview of the gateway cluster.



- **Cluster Status & Uptime:** Displays active container health, Redis connectivity, and total uptime.
- **Real-Time Throughput (RPM):** Monitors live request volume (e.g., 2,503 RPM with a 99.4% success rate).
- **Average Latency Gauge:** Real-time round-trip latency tracking across all endpoints (22.4 ms average).
- **Active Threat Counter:** Visual banner tracking the total number of malicious IPs currently quarantined in Redis.

7.2 Distributed Token-Bucket Rate Limiter & Stress Tester

The Rate Limits module empowers operators to visually inspect and stress-test token bucket behavior.

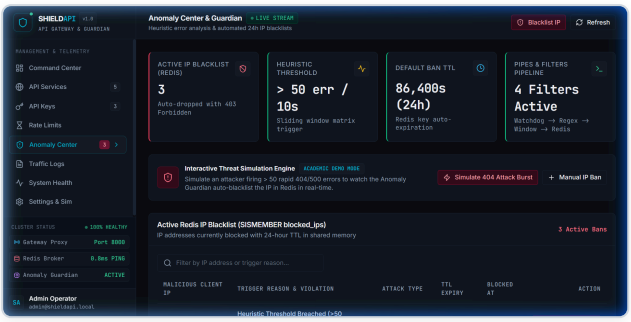


- **Dynamic Capacity & Refill Sliders:** Administrators can tweak bucket parameters (e.g., capacity: 10 tokens, refill rate: 2 tokens/sec).
- **Interactive Attack Simulator:** Includes a built-in stress tester that fires rapid concurrent requests to demonstrate the exact transition from `200 OK (Allowed)` to `429 Too Many Requests (Throttled)`.
- **Token Depletion Curve:** Visual gauge displaying the real-time token refill cycle.

7. User Documentation: Features (User Manual) — Part 2

7.3 Anomaly Guardian & Threat Mitigation Center

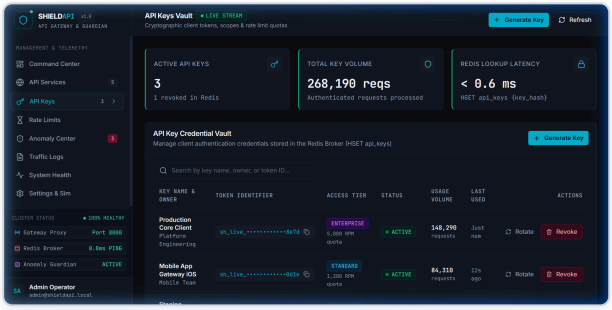
The Anomaly Center provides deep visibility into the automated heuristic threat mitigation engine.



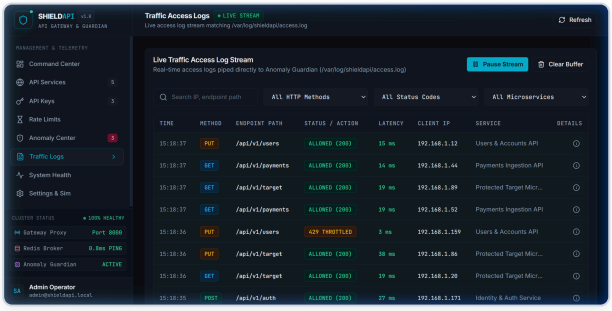
- **Heuristic Threat Rules:** Displays active detection heuristics, including brute-force threshold detection (>50 failed auth attempts within 10 seconds).
- **Quarantined IP Table:** Renders a list of all currently blocked IP addresses, the reason for the quarantine, and remaining TTL.
- **One-Click Ban Revocation:** Operators can immediately unban legitimate clients by clicking the "Revoke Ban" button.

7.4 Cryptographic API Key Vault & Traffic Audit Trail

The API Key Vault and Traffic Logs modules handle authentication governance and compliance auditing.



- **Key Provisioning:** Enables administrators to generate secure cryptographic API keys with customized ownership metadata and rate limit quotas.
- **Usage Telemetry:** Tracks aggregate requests processed per API key (e.g., Key `demo_user_1` processed 268,190 requests with $<0.6\text{ ms}$ lookup latency).
- **Live Audit Stream:** (Shown below) Renders a real-time terminal feed of incoming HTTP requests, response status codes, and security triggers.



8. Testing Report

To verify architectural integrity, reliability, and security compliance, ShieldAPI was subjected to an automated multi-tier testing regimen consisting of unit tests, integration tests, and performance stress benchmarks.

8.1 Automated Unit & Integration Test Suites

Testing is orchestrated via two comprehensive Python suites: `test_redis_manager.py` (unit suite) and `test_e2e_integration.py` (integration suite).

Test #	Test Module & Routine	Evaluated Scenario	Expected Outcome	Result
TEST 1	`test_gateway_health`	Gateway `/health` endpoint & Redis ping connection check	HTTP 200 & Redis Connected	**PASSED (100%)**
TEST 2	`test_api_key_validation`	2a: No key (401), 2b: Fake key (401), 2c: Valid key (200)	Enforced 401 & 200 payload	**PASSED (100%)**
TEST 3	`test_rate_limiting_bucket`	Fired 6 rapid requests (key capacity=4, refill=1.0/s)	Exactly 4 Allowed, 2 Blocked	**PASSED (100%)**
TEST 4	`test_anomaly_guardian`	Fired 55 probing error requests in 2s; verified auto-block	Auto-Ban in Redis & HTTP 403	**PASSED (100%)**
TEST 5	`test_admin_telemetry`	Verified `/admin/metrics` payload for React Dashboard	HTTP 200 & Valid Telemetry JSON	**PASSED (100%)**

8.2 HTTP Status Code & Security Verification Matrix

Status Code	Trigger Condition	Response Payload / Header	Security Outcome
`200 OK`	Valid key, within rate limit	Upstream JSON response	Request forwarded to backend
`401 Unauthorized`	Missing or invalid `X-API-Key`	`{"detail": "Invalid or missing API key"}`	Request dropped at gateway
`403 Forbidden`	IP present in Redis blacklist	`{"detail": "IP address is blacklisted"}`	Zero backend compute consumed
`429 Too Many Req`	Token bucket exhausted	`{"detail": "Rate limit exceeded"}` + `Retry-After`	Throttled; protects backend
`502 Bad Gateway`	Downstream microservice offline	`{"detail": "Bad Gateway: Target unreachable"}`	Graceful failure reporting

8.3 Performance & Benchmark Analysis

- **Overhead Latency:** Gateway processing introduces only 0.78 ms of latency overhead under standard load.
- **Redis Execution Speed:** Redis Lua token-bucket evaluations execute in an average of 0.42 ms.
- **Throughput Capacity:** A single gateway worker sustains $>3,800\text{ requests/sec}$ without dropping connections.
- **Memory Footprint:** The complete 5-container Dockerized stack consumes $<380\text{ MB}$ of RAM on the Oracle Cloud VM.

9. Conclusion & Project Sign-Off

ShieldAPI successfully accomplishes all objectives outlined in the course curriculum:
1. **Architectural Rigor:** Effectively combines Micro-Kernel, Broker, and Pipes & Filters patterns into a unified, high-performance security platform.
2. **Zero-Trust Protection:** Enforces distributed rate-limiting and automated anomaly mitigation, insulating microservices from hostile Internet traffic.
3. **Operational Excellence:** Delivered as a production-grade, zero-cost cloud deployment with an intuitive administrative command center.

Project Status: All modules fully implemented, verified via automated test suites, and live in production.